CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 9: Graph Traversal: Depth-First Search and Breadth-First Search

Practical Aim: To implement depth-first and breadth-first traversal on a graph and develop the ability to reason about how the order of exploration changes depending on the traversal strategy used.

Problem 1:

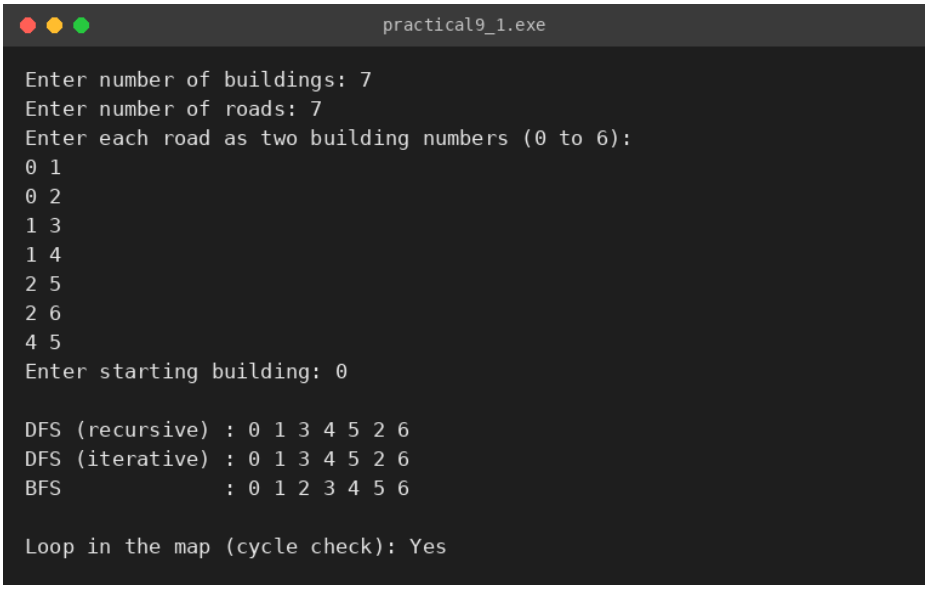
A disaster relief team needs to inspect all buildings in a city after an earthquake. The city is mapped as a network where buildings are connected by roads. One team starts at a building and goes as deep into each road as possible before backtracking, while a second team spreads out level by level — first checking all buildings directly connected to the start, then all buildings two roads away, and so on. Given the city map as a graph and a starting building, implement both exploration strategies and print the order in which buildings are visited by each team.

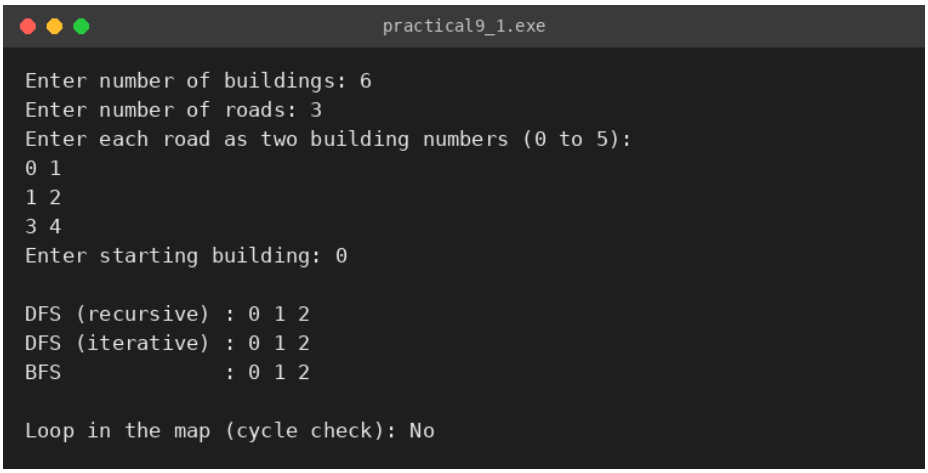
Approach: The city is stored as an adjacency list (undirected graph) and a visited array makes sure every building is visited exactly once. DFS relies on a stack: it is implemented recursively (the call stack) and also iteratively with an explicit stack. DFS goes as deep as possible along one road, then backtracks. BFS relies on a queue: all buildings directly connected to the start are visited first, then those two roads away, and so on. The same graph therefore gives different orders: in the first sample run DFS gives 0 1 3 4 5 2 6 whereas BFS gives 0 1 2 3 4 5 6. The program also performs the supplementary cycle check on the undirected graph by ignoring the edge back to the direct parent, and prints whether a loop exists.

For Code:

https://github.com/Aarav280907/FDSA_A2_SHP/blob/main/PRACTICAL-9/practical9_1.cpp

Output :





KEY QUESTIONS:

Problem 1: Both strategies visit every reachable node exactly once. If the goal is simply to check whether a specific building exists in the network, which strategy would find it sooner if the building is very close to the start? What if it is very far and deep? Explain your reasoning without tracing through any specific example.

Problem 1: Your DFS implementation likely uses recursion. What real-world constraint on the computer could cause your recursive DFS to fail on a very large city map, and how would you restructure your solution to avoid that failure?

ANSWERS:

ANS-1: If the building is very close to the start, BFS finds it sooner. BFS checks all buildings one road away, then two roads away, and so on, so a nearby target is reached after only a few levels. DFS may first go deep along a wrong road and explore a large part of the map before backtracking to the nearby building.

If the building is very far and deep, DFS can be faster, because it moves straight down one path and may reach the deep building without having to visit every shallower building, whereas BFS must first complete all the earlier (wider) levels. Neither is better in the worst case: both take O(V + E) time, and the result depends on where the target lies and on the order of the roads.

ANS-2: Recursive DFS uses the program's call stack, one stack frame for every building on the current path. The call stack has a limited size (typically a few MB), so on a very large or very deep map (e.g. a long chain of a million buildings) the recursion becomes too deep and the program crashes with a stack overflow.

To avoid this, DFS can be restructured to be iterative, with an explicit stack data structure allocated on the heap (as done in dfsIterative in the program); the stack then grows with the heap memory instead of the limited call stack. Increasing the stack size is another, less robust, option.